

Modern CSS & layout (Flexbox/Grid)

COMPLETE LEARNING GUIDE

What You'll Learn

Before Flexbox and Grid, CSS layout relied on hacks — floats meant for wrapping text around images, or table-based layouts — that never behaved intuitively for full-page structure. Flexbox (2017-ish widespread adoption) and CSS Grid solved this properly: Flexbox distributes items along a single axis and excels at things like navigation bars, button groups, and centering; Grid defines both rows and columns simultaneously and excels at full page and component layouts with precise two-dimensional placement. This submodule covers both systems in depth — the core vocabulary of main/cross axis in Flexbox, the track/area vocabulary of Grid, the alignment properties they share, and the practical judgment call of when to reach for one over the other. It also covers the box model foundation both systems sit on top of, since a surprising number of 'Flexbox bugs' are actually box-sizing bugs in disguise.

1. The Box Model and box-sizing

Every element generates a box made of content, padding, border, and margin, stacked from the inside out. By default (content-box), the width and height you set apply only to the content area — adding 20px of padding and a 2px border to an element with width: 200px makes it render at 244px total, which surprises almost everyone the first time they hit it. Setting box-sizing: border-box changes width/height to include padding and border, so a 200px-wide box with padding stays 200px wide, with the content area shrinking instead. Nearly every modern CSS reset applies border-box globally as the very first rule, because it matches how most developers actually reason about sizing.

CSS

```
*, *::before, *::after {
  box-sizing: border-box;
}

.card {
  width: 300px;
  padding: 1.5rem;
  border: 1px solid #ddd;
  /* renders at exactly 300px wide, not 300 + padding + border */
}
```

2. Flexbox: One-Dimensional Layout

Flexbox lays out children of a flex container along a single main axis (row by default, or column with `flex-direction: column`), with a perpendicular cross axis. `justify-content` aligns items along the main axis (`flex-start`, `center`, `space-between`, `space-around`, `space-evenly`); `align-items` aligns items along the cross axis (`stretch` is the default, plus `center`, `flex-start`, `flex-end`). Individual items grow and shrink to fill or fit available space via `flex-grow`, `flex-shrink`, and `flex-basis` — almost always written as the flex shorthand (`flex: 1 1 0` or simply `flex: 1` to mean 'grow and shrink equally, ignore intrinsic size'). `flex-wrap: wrap` lets items flow onto new lines instead of being forced to shrink indefinitely or overflow.

- `justify-content` = main axis alignment; `align-items` = cross axis alignment — mixing these up is the single most common Flexbox confusion.
- `flex: 1` on every child of a flex container makes them share available space equally regardless of content length.
- `gap` (not margin hacks) is the modern way to add spacing between flex items, and it doesn't add extra space at the container's edges.
- `align-self` overrides `align-items` for one individual flex item.

CSS

```
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  padding: 0.75rem 1.5rem;
}

.navbar__links {
  display: flex;
  gap: 1.5rem;
}

.card-row {
  display: flex;
  flex-wrap: wrap;
  gap: 1rem;
}

.card-row > .card {
  flex: 1 1 250px; /* grow/shrink, but never smaller than ~250px before wrapping */
}
```

3. CSS Grid: Two-Dimensional Layout

Grid defines an explicit set of rows and columns and places items into that structure directly, which makes it the natural choice for full-page layouts and any component that needs precise control over both dimensions at once. `grid-template-columns` and `grid-template-rows` define track sizes using fixed lengths, percentages, or the `fr` unit (a share of remaining space after fixed tracks are subtracted). Items are placed automatically in source order, or explicitly with `grid-column`/`grid-row` line numbers, or — often the most readable option — named

template areas that let you sketch the layout visually right in the CSS.

CSS

```
.page {
  display: grid;
  grid-template-columns: 240px 1fr;
  grid-template-areas:
    "sidebar header"
    "sidebar main";
  min-height: 100vh;
}
.sidebar { grid-area: sidebar; }
.header { grid-area: header; }
.main { grid-area: main; }

/* A responsive card grid with NO media queries at all */
.gallery {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 1.5rem;
}
```

4. auto-fill vs. auto-fit with minmax()

The `repeat(auto-fit, minmax(220px, 1fr))` pattern is one of the most powerful tools in modern CSS: it tells the browser to fit as many 220px-minimum columns as will comfortably fit the container, and to stretch them to fill any remaining space — producing a fully responsive grid without writing a single media query. `auto-fill` and `auto-fit` differ subtly: `auto-fill` keeps empty tracks (leaving gaps if there aren't enough items to fill a row), while `auto-fit` collapses empty tracks to zero width, letting existing items stretch to fill the leftover space. For a card gallery where you always want items to fill the available width, `auto-fit` is almost always the right choice.

5. Shared Alignment Properties: `place-items`, `gap`, `justify-self`

Grid and Flexbox share much of their alignment vocabulary, which makes the two systems easier to learn together than separately. `gap` works identically in both to add consistent spacing between items without margin hacks. `justify-self`/`align-self` position an individual item within its own grid cell or flex line, mirroring `justify-content`/`align-items` at the container level. The `place-items` and `place-content` shorthands set both the row and column (or main and cross axis) alignment in a single declaration — `place-items: center center` is the single most common and reliable way to center anything, replacing older hacks involving absolute positioning and negative margins.

CSS

```
.centered-box {  
  display: grid;  
  place-items: center; /* centers child both horizontally and vertically */  
  min-height: 100vh;  
}
```

6. Choosing Flexbox vs. Grid

The practical rule: reach for Flexbox when you're arranging items along a single row or column and want them to react naturally to their own content size (a nav bar, a button group, a horizontally scrolling list of tags). Reach for Grid when you're laying out a structure with meaningful rows and columns at the same time (a page skeleton, a photo gallery, a dashboard of cards, form layouts with aligned labels and inputs). In practice most real interfaces use both together — Grid for the outer page/component skeleton, Flexbox for the one-dimensional arrangements nested inside each grid cell.

- One dimension, content-driven sizing !' Flexbox.
- Two dimensions, structure-driven sizing !' Grid.
- It's completely normal (and common) to nest a flex container inside a grid item, or vice versa.

7. Common Flex/Grid Sizing Gotchas

Flex and grid items have a default min-width/min-height of auto, which means they won't shrink below their content's intrinsic size by default — this is the root cause behind long unbreakable text or a wide image overflowing a flex/grid container even though flex-shrink is set. Setting min-width: 0 (for row layouts) or min-height: 0 (for column layouts) on the item removes that floor and lets it shrink or truncate (with overflow: hidden and text-overflow: ellipsis) as expected. This single property fixes a large fraction of 'my flex item won't shrink and it's overflowing the page' bug reports.

CSS

```
.flex-row { display: flex; gap: 1rem; }  
.flex-row .text-cell {  
  min-width: 0; /* allow shrinking below content's intrinsic width */  
  overflow: hidden;  
  text-overflow: ellipsis;  
  white-space: nowrap;  
}
```

Common Pitfalls

- Forgetting box-sizing: border-box, then being confused why padding/border makes elements wider than the declared width.

- Confusing justify-content (main axis) with align-items (cross axis), especially after switching flex-direction to column.
- Assuming flex items shrink freely — they respect a default min-width: auto floor based on content, which overflow: hidden alone doesn't fix without also setting min-width: 0.
- Using margin on individual flex/grid children to fake spacing instead of the container's gap property, leading to inconsistent edge spacing.
- Reaching for absolute positioning and negative margins to center content instead of place-items: center or margin: auto in a flex container.
- Using floats and clearfix hacks for layout in new code when Flexbox/Grid make them unnecessary.
- Not realizing auto-fill leaves empty gap tracks while auto-fit collapses them — picking the wrong one gives a layout that looks broken with few items.

Key Takeaways

- box-sizing: border-box should be a global reset in nearly every project — it makes width/height match real developer intuition.
- Flexbox = one axis at a time; Grid = both axes simultaneously — pick based on whether the layout is fundamentally 1D or 2D.
- repeat(auto-fit, minmax(min, 1fr)) builds a fully responsive grid without a single media query.
- gap is the modern, correct way to space out flex/grid children — stop reaching for margin hacks.
- min-width: 0 / min-height: 0 is the fix for flex/grid items that refuse to shrink below their content size.

Learn More

CSS-Tricks — A Complete Guide to Flexbox

<https://css-tricks.com/snippets/css/a-guide-to-flexbox/>

CSS-Tricks — A Complete Guide to Grid

<https://css-tricks.com/snippets/css/complete-guide-grid/>

MDN — CSS Grid Layout

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_grid_layout